

OS Lab6实验报告

练习0：填写已有实验

1.proc.c 中 alloc_proc() 的更新

在 Lab6 中，进程控制块 `proc_struct` 新增了调度相关字段，需要在 `alloc_proc()` 中初始化：

```
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        // ... Lab4/5 的初始化代码 ...

        // LAB6: 新增调度相关字段的初始化
        proc->rq = NULL;                // 运行队列指针初始化为空
        list_init(&(proc->run_link));    // 初始化运行队列链表节点
        proc->time_slice = 0;           // 时间片初始化为0（入队时设置）
        skew_heap_init(&(proc->lab6_run_pool)); // 初始化Stride调度的斜堆节点
        proc->lab6_stride = 0;          // stride值初始化为0
        proc->lab6_priority = 16;       // 默认优先级设为16
    }
    return proc;
}
```

新增字段说明：

字段	类型	用途
rq	struct run_queue *	指向进程所属的运行队列
run_link	list_entry_t	RR调度中链接到运行队列的节点
time_slice	int	剩余时间片
lab6_run_pool	skew_heap_entry_t	Stride调度中的斜堆节点
lab6_stride	uint32_t	当前stride值（累计步进）
lab6_priority	uint32_t	进程优先级

2. trap.c 中时钟中断处理的更新

Lab6 的时钟中断需要调用调度器的 `proc_tick` 函数来更新进程时间片：

```
// 在文件开头声明外部函数
extern void sched_class_proc_tick(struct proc_struct *proc);

// 在 interrupt_handler() 的 IRQ_S_TIMER 分支中更新
case IRQ_S_TIMER:
    // 设置下一次时钟中断
```

```

clock_set_next_event();

// 全局ticks计数器增加
if (++ticks % TICK_NUM == 0) {
    print_ticks(); // 保留LAB3的100 ticks打印逻辑
}

// LAB6: 调度器核心更新 - 每个时钟中断更新当前进程的调度信息
if (current != NULL) {
    sched_class_proc_tick(current);
}
break;

```

与 Lab5 的区别:

方面	Lab5	Lab6
时钟中断处理	仅计数 ticks	调用 sched_class_proc_tick()
调度触发	无自动触发	时间片耗尽自动设置 need_resched

3. 编译验证

完成代码迁移后, 执行以下命令验证:

```

$ make clean
$ make
$ make qemu

```

输出包含:

- sched class: RR_scheduler - 表示调度器初始化成功
- 每 100 ticks 打印 100 ticks
- 用户进程能够正常调度执行

表明我们成功完成练习0!

练习1: 理解调度器框架的实现 (不需要编码)

一、调度类结构体 sched_class 的分析

1.1 sched_class 结构体定义

```

struct sched_class {
    const char *name; // 调度类名称
    void (*init)(struct run_queue *rq); // 初始化运行队列
    void (*enqueue)(struct run_queue *rq, struct proc_struct *proc); // 入队
    void (*dequeue)(struct run_queue *rq, struct proc_struct *proc); // 出队
    struct proc_struct *(*pick_next)(struct run_queue *rq); // 选择下一个进程
    void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc); // 时钟处理

```

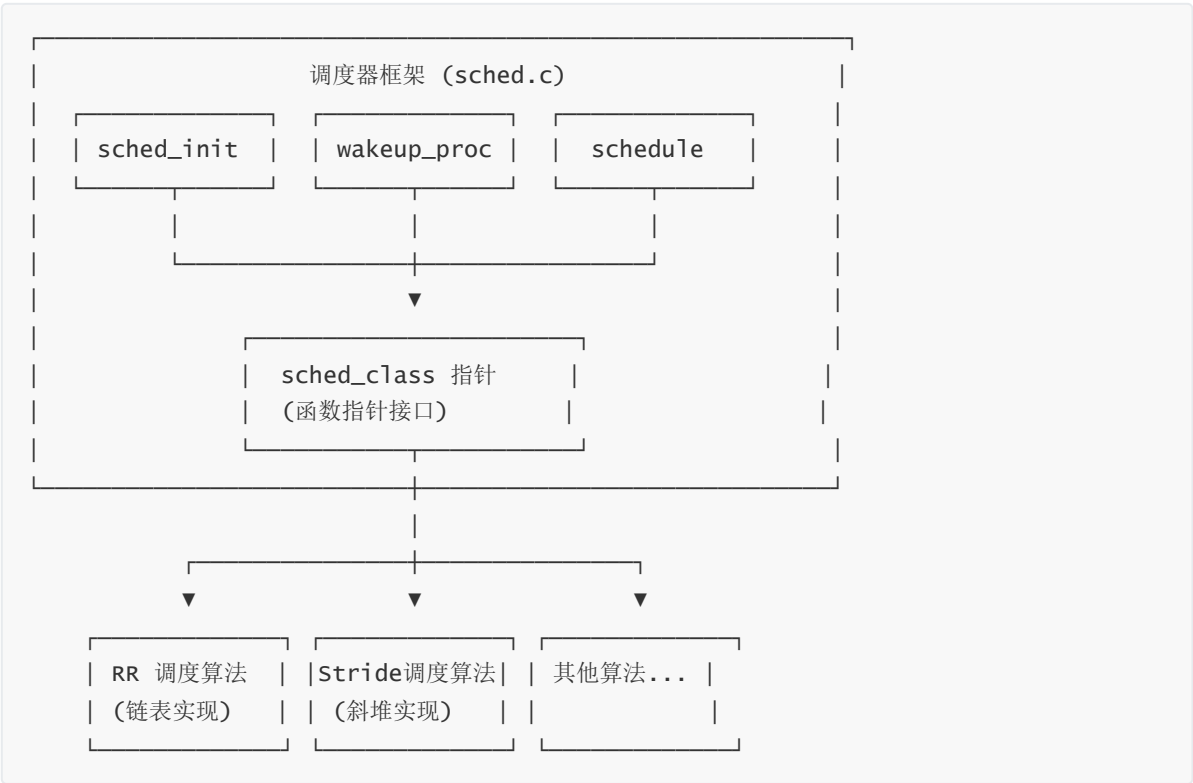
```
};
```

1.2 各函数指针的作用和调用时机

函数指针	作用	调用时机	调用位置
init	初始化运行队列的数据结构 (链表或斜堆)	系统启动时	sched_init()
enqueue	将进程加入就绪队列，设置 时间片	唤醒进程或当前进 程让出CPU	wakeup_proc(), schedule()
dequeue	将进程从就绪队列移除	选中进程准备运行 时	schedule()
pick_next	从就绪队列选择下一个要运 行的进程	调度决策时	schedule()
proc_tick	处理时钟中断，更新时间片 或stride	每次时钟中断	sched_class_proc_tick()

1.3 为什么使用函数指针而非直接实现？

设计模式：策略模式（Strategy Pattern）



使用函数指针的优势：

- 解耦合**：框架代码不依赖具体算法实现，只依赖接口
- 可扩展性**：添加新算法只需实现接口，无需修改框架
- 运行时切换**：修改 `sched_class` 指针即可切换算法
- 代码复用**：中断处理、进程管理等代码被所有算法共享

二、运行队列结构体 run_queue 的分析

2.1 Lab5 vs Lab6 的 run_queue 对比

Lab5 没有独立的 run_queue 结构，直接使用全局 `proc_list` 遍历查找可运行进程。

Lab6 的 run_queue 定义：

```
struct run_queue {
    list_entry_t run_list;           // 双向链表（用于RR调度）
    unsigned int proc_num;          // 就绪进程数量
    int max_time_slice;             // 最大时间片
    skew_heap_entry_t *lab6_run_pool; // 斜堆根节点（用于Stride调度）
};
```

2.2 详细对比

特性	Lab5	Lab6
数据结构	直接遍历 proc_list	独立的 run_queue
就绪队列	无专门队列	run_list 链表
优先级支持	无	lab6_run_pool 斜堆
进程计数	无	proc_num
时间片管理	无	max_time_slice
调度复杂度	O(n) 遍历所有进程	O(1) 或 O(log n)

2.3 为什么需要两种数据结构？

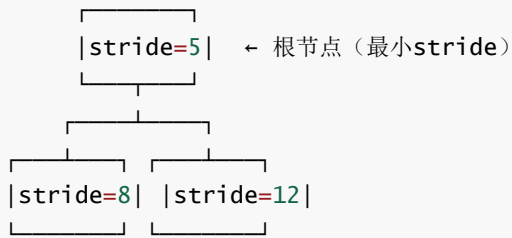
1.双向链表 (list_entry_t run_list) - 用于 RR 调度

```
// 链表结构示意图
run_list ↔ proc1.run_link ↔ proc2.run_link ↔ proc3.run_link ↔ run_list
 头节点      队首           中间           队尾           回到头
```

- **时间复杂度：**入队 O(1)，出队 O(1)，选择 O(1)
- **适用场景：**所有进程平等，按 FIFO 顺序轮转
- **实现简单，**内存开销小

2.斜堆 (skew_heap_entry_t lab6_run_pool) - 用于 Stride 调度

```
// 斜堆结构（最小堆，stride最小的在根）
struct skew_heap_entry {
    struct skew_heap_entry *parent, *left, *right;
};
```



- **时间复杂度**：插入 $O(\log n)$ ，删除 $O(\log n)$ ，取最小 $O(1)$
- **适用场景**：需要快速找到 stride 最小（优先级最高）的进程
- **自平衡**：通过合并操作维持堆性质

斜堆核心操作分析：

```
// 合并操作 - 斜堆的核心
static inline skew_heap_entry_t *
skew_heap_merge(skew_heap_entry_t *a, skew_heap_entry_t *b, compare_f comp) {
    if (a == NULL) return b;
    if (b == NULL) return a;

    if (comp(a, b) == -1) {          // a 的 stride 更小
        // a 作为根，交换 a 的左右子树，递归合并
        r = a->left;
        l = skew_heap_merge(a->right, b, comp);
        a->left = l;
        a->right = r;
        return a;
    } else {                        // b 的 stride 更小
        // b 作为根，类似处理
        ...
    }
}
```

三、调度器框架函数分析（Lab5 vs Lab6）

3.1 wakeup_proc() 函数对比

Lab5 实现：

```

void wakeup_proc(struct proc_struct *proc) {
    assert(proc->state != PROC_ZOMBIE);
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        if (proc->state != PROC_RUNNABLE) {
            proc->state = PROC_RUNNABLE;
            proc->wait_state = 0;
            // 没有将进程加入就绪队列
        }
    }
    local_intr_restore(intr_flag);
}

```

Lab6 实现:

```

void wakeup_proc(struct proc_struct *proc) {
    assert(proc->state != PROC_ZOMBIE);
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        if (proc->state != PROC_RUNNABLE) {
            proc->state = PROC_RUNNABLE;
            proc->wait_state = 0;
            if (proc != current) {
                sched_class_enqueue(proc); // 通过调度类接口入队
            }
        }
    }
    local_intr_restore(intr_flag);
}

```

关键变化: Lab6 通过 `sched_class_enqueue()` 将进程加入就绪队列，实现了与具体调度算法的解耦。

3.2 schedule() 函数对比

Lab5 实现 (硬编码遍历) :

```

void schedule(void) {
    // ...
    // 直接遍历 proc_list 查找可运行进程
    last = (current == idleproc) ? &proc_list : &(current->list_link);
    le = last;
    do {
        if ((le = list_next(le)) != &proc_list) {
            next = le2proc(le, list_link);
            if (next->state == PROC_RUNNABLE) {
                break; // 找到第一个可运行进程
            }
        }
    } while (le != last);
    // ...
}

```

```
}
```

Lab6 实现（通过调度类接口）：

```
void schedule(void) {
    // ...
    current->need_resched = 0;

    // 1. 当前进程重新入队（如果仍可运行）
    if (current->state == PROC_RUNNABLE) {
        sched_class_enqueue(current);    // 调用 sched_class->enqueue
    }

    // 2. 选择下一个进程
    if ((next = sched_class_pick_next()) != NULL) { // 调用 sched_class->pick_next
        sched_class_dequeue(next);        // 调用 sched_class->dequeue
    }

    // 3. 无就绪进程则运行 idle
    if (next == NULL) {
        next = idleproc;
    }
    // ...
}
```

关键变化：

方面	Lab5	Lab6
调度决策	硬编码在 schedule() 中	委托给 sched_class
就绪队列	遍历全局 proc_list	使用独立的 run_queue
可扩展性	修改调度需改 schedule()	只需实现新的 sched_class

3.3 sched_init() 函数（Lab6 新增）

```
void sched_init(void) {
    list_init(&timer_list);    // 初始化定时器
    sched_class = &default_sched_class; // 绑定调度类
    rq = &__rq;                // 设置运行队列
    rq->max_time_slice = MAX_TIME_SLICE; // 设置时间片（默认5）
    sched_class->init(rq);      // 调用具体算法初始化
    cprintf("sched class: %s\n", sched_class->name);
}
```

4.1 完整初始化流程图

4.2 default_sched_class 与框架的关联

```
// default_sched.h 中声明
extern struct sched_class default_sched_class;
extern struct sched_class stride_sched_class;
```

```
// default_sched.c 中定义
struct sched_class default_sched_class = {
    .name = "RR_scheduler",
    .init = RR_init,
    .enqueue = RR_enqueue,
    .dequeue = RR_dequeue,
    .pick_next = RR_pick_next,
    .proc_tick = RR_proc_tick,
};

// sched.c 中关联
static struct sched_class *sched_class; // 全局调度类指针
sched_class = &default_sched_class;    // 绑定到 RR 调度
```

五、进程调度流程

5.1 完整调度流程

当IRQ_S_TIMER时钟中断发生时，会执行trap.c中的中断处理流程。

首先，中断处理函数会：

1. 调用clock_set_next_event()设置下一次时钟中断
2. 将全局ticks计数器加1
3. 调用sched_class_proc_tick(current)处理当前进程的时钟中断

在sched_class_proc_tick函数中：

- 如果当前进程不是空闲进程，则调用具体的调度类proc_tick方法（如RR_proc_tick）
- 如果是空闲进程，则直接设置need_resched标志为1，强制让出CPU

对于RR（轮询）调度算法，RR_proc_tick的处理逻辑是：

1. 如果进程的时间片大于0，则将时间片减1
2. 如果时间片减到0，则设置该进程的need_resched标志为1

当trap中断处理返回时（仅从用户态返回时）：

- 检查当前进程的need_resched标志
- 如果标志为真且不在内核态，则调用schedule()函数触发调度

schedule()调度函数的执行步骤：

1. **清除调度标志**：将当前进程的need_resched设为0
2. **处理当前进程**：如果当前进程状态仍为可运行（PROC_RUNNABLE）
 - 调用调度类的enqueue方法（如RR_enqueue）将当前进程放回就绪队列
 - RR_enqueue具体操作：
 - 将进程插入就绪队列的链表
 - 如果时间片为0，则重置为最大时间片
 - 设置进程的运行队列指针
 - 增加运行队列的进程计数
3. **选择下一个进程**：调用调度类的pick_next方法（如RR_pick_next）

- RR_pick_next具体操作：
 - 从就绪队列链表中选择下一个进程
 - 如果队列不为空，返回下一个进程
 - 否则返回NULL

4. 从队列中移除：如果选出了下一个进程

- 调用调度类的dequeue方法（如RR_dequeue）
- RR_dequeue具体操作：
 - 从就绪队列链表中移除该进程
 - 减少运行队列的进程计数

5. 处理无就绪进程情况：如果没有可选的就绪进程

- 将下一个进程设置为空闲进程（idleproc）

6. 执行上下文切换：

- 增加选中进程的运行次数计数
- 如果选中的进程不是当前进程，则调用proc_run()进行上下文切换

整个过程实现了基于时间片的抢占式调度，当时钟中断发现进程时间片用完时，通过设置调度标志，最终在中断返回时触发完整的调度过程。

5.2 need_resched 标志位的作用

阶段	need_resched	说明
进程正常运行	0	不需要调度，继续执行
时间片耗尽	→ 1	RR_proc_tick 中设置
进程主动让出	→ 1	do_yield() 中设置
idle 进程	始终 1	sched_class_proc_tick 中强制设置
schedule() 开始	→ 0	清除标志，开始调度

延迟调度机制：

- 在中断处理中只设置标志，不立即调度
- 中断返回用户态时检查标志并执行调度
- 避免在中断上下文中进行复杂的调度操作

六、调度算法的切换机制

6.1 添加新调度算法的步骤

以添加 Stride 调度为例：

步骤1：创建新的调度类文件 `stride_sched.c`

```
#include <proc.h>
#include <sched.h>
```

```

#define BIG_STRIDE 0x7FFFFFFF

// 比较函数: stride 小的优先级高
static int proc_stride_comp(void *a, void *b) {
    struct proc_struct *pa = le2proc(a, lab6_run_pool);
    struct proc_struct *pb = le2proc(b, lab6_run_pool);
    int32_t c = pa->lab6_stride - pb->lab6_stride;
    if (c > 0) return 1;
    else if (c == 0) return 0;
    else return -1;
}

static void stride_init(struct run_queue *rq) {
    list_init(&(rq->run_list));
    rq->lab6_run_pool = NULL;
    rq->proc_num = 0;
}

static void stride_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    // 插入斜堆
    rq->lab6_run_pool = skew_heap_insert(
        rq->lab6_run_pool, &(proc->lab6_run_pool), proc_stride_comp);
    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice)
        proc->time_slice = rq->max_time_slice;
    proc->rq = rq;
    rq->proc_num++;
}

static void stride_dequeue(struct run_queue *rq, struct proc_struct *proc) {
    // 从斜堆删除
    rq->lab6_run_pool = skew_heap_remove(
        rq->lab6_run_pool, &(proc->lab6_run_pool), proc_stride_comp);
    rq->proc_num--;
}

static struct proc_struct *stride_pick_next(struct run_queue *rq) {
    if (rq->lab6_run_pool == NULL) return NULL;
    // 堆顶即为 stride 最小的进程
    struct proc_struct *p = le2proc(rq->lab6_run_pool, lab6_run_pool);
    // 更新 stride
    p->lab6_stride += BIG_STRIDE / p->lab6_priority;
    return p;
}

static void stride_proc_tick(struct run_queue *rq, struct proc_struct *proc) {
    if (proc->time_slice > 0)
        proc->time_slice--;
    if (proc->time_slice == 0)
        proc->need_resched = 1;
}

struct sched_class stride_sched_class = {
    .name = "stride_scheduler",
    .init = stride_init,
    .enqueue = stride_enqueue,
    .dequeue = stride_dequeue,

```

```
.pick_next = stride_pick_next,  
.proc_tick = stride_proc_tick,  
};
```

步骤2: 修改 sched_init() 中的调度类指针

```
void sched_init(void) {  
    list_init(&timer_list);  
  
    // 方法1: 直接切换  
    // sched_class = &default_sched_class; // RR 调度  
    sched_class = &stride_sched_class;    // Stride 调度  
  
    // 方法2: 编译时选择  
    #ifdef USE_STRIDE_SCHED  
        sched_class = &stride_sched_class;  
    #else  
        sched_class = &default_sched_class;  
    #endif  
  
    rq = &__rq;  
    rq->max_time_slice = MAX_TIME_SLICE;  
    sched_class->init(rq);  
}
```

6.2 需要修改的代码汇总

修改内容	文件	修改量
实现新调度类	stride_sched.c (新建)	约100行
声明调度类	default_sched.h	1行
切换调度类指针	sched.c 的 sched_init()	1行
框架代码	无需修改	0

6.3 为什么切换调度算法容易?

1. 接口统一

- 所有调度算法实现相同的 sched_class 接口
- 框架只依赖接口, 不依赖具体实现

2. 单点切换

- 只需修改 sched_class 指针的赋值
- 一行代码完成算法切换

3. 数据结构兼容

- run_queue 同时支持链表和斜堆
- proc_struct 包含两种数据结构的节点

4. 无侵入性修改

- 新算法不需要修改：
 - schedule() 函数
 - wakeup_proc() 函数
 - trap.c 中断处理
 - proc.c 进程管理

5. 编译时/运行时切换

- 编译时：通过宏定义选择
- 运行时：直接修改 sched_class 指针

练习2：实现 Round Robin 调度算法（需要编码）

2.1 Lab5与Lab6中实现不同的函数分析

2.1.1 wakeup_proc() 函数的变化

```
void wakeup_proc(struct proc_struct *proc) {
    assert(proc->state != PROC_ZOMBIE && proc->state != PROC_RUNNABLE);
    proc->state = PROC_RUNNABLE;
}

void wakeup_proc(struct proc_struct *proc) {
    assert(proc->state != PROC_ZOMBIE);
    bool intr_flag;
    local_intr_save(intr_flag);
    {
        if (proc->state != PROC_RUNNABLE) {
            proc->state = PROC_RUNNABLE;
            proc->wait_state = 0;
            if (proc != current) {
                sched_class_enqueue(proc);
            }
        }
        else {
            warn("wakeup runnable process.\n");
        }
    }
    local_intr_restore(intr_flag);
}
```

变化原因：

Lab6 要实现 **Round Robin（轮询调度）**，而轮询调度的核心是“有序管理可运行进程队列”，因此需要对 wakeup_proc() 做针对性改造：

1. 调度机制从“遍历扫描”升级为“队列管理”

1. Lab5 中 schedule() 会扫描所有进程结构体，筛选 state=PROC_RUNNABLE 的进程。因此 wakeup_proc 只需修改状态，schedule 就能通过遍历找到它。
2. Lab6 的 Round Robin 调度依赖“可运行队列”，调度器需要按“队列顺序”为进程分配时间片，因此必须将唤醒的进程加入可运行队列

2. 临界区保护：避免队列操作的“竞态条件”

1. 若操作队列时被中断打断，会导致队列链表的指针不一致，最终队列结构损坏。
2. Lab6 用 `local_intr_save/restore` 关闭 / 恢复中断，保证队列操作的原子性；Lab5 无队列操作，因此不需要。

3. 清理等待状态：修复进程状态管理混乱

1. 唤醒时必须重置 `wait_state=0`，标记进程“脱离等待状态”；否则后续进程再次等待时，会因“旧等待标记未清”出现状态冲突。
2. Lab5 未设计 `wait_state` 机制，因此无需处理。

4. 不做改动会出现的问题：

1. 若不将进程加入可运行队列，Lab6 的 `schedule` 是从“可运行队列”中取进程，即使进程 `state=PROC_RUNNABLE`，只要不在队列中，调度器就找不到它——进程会永远得不到 CPU 时间，直接“饿死”。
2. 若不添加中断保护，操作队列时被中断打断，会导致队列链表结构损坏如指针错乱、节点丢失，后续 `schedule` 操作队列时会出现野指针、重复调度等问题，严重时直接导致系统崩溃。
3. 若不重置 `wait_state`：进程被唤醒后仍保留“等待标记”，后续进入等待状态时会出现冲突，导致进程无法正确响应资源请求，引发死锁或状态异常。

2.2 RR调度算法各函数的实现

2.2.1 RR_init 函数实现

```
static void
RR_init(struct run_queue *rq)
{
    // 初始化运行队列为空链表
    list_init(&(rq->run_list));
    // 初始化进程计数为0
    rq->proc_num = 0;
    // max_time_slice 由调用者设置，此处不需要初始化
}
```

设计思路：

- `rq->run_list` 是就绪队列的哨兵节点，不存储实际进程数据，仅用于简化链表操作。`list_init` 将其 `prev` 和 `next` 指针均指向自身，此时链表处于“空”状态，所有遍历操作从哨兵节点开始，若后继节点仍是哨兵节点，则说明队列无有效进程。哨兵节点的设计避免了普通链表空队列判断头尾插入逻辑不一致的问题，是 ucore 链表的核心特性
- `proc_num` 是就绪队列的进程计数器，初始化为 0 表示队列中暂无就绪进程。该字段用于快速获取队列长度，避免每次遍历链表统计数量，提升调度效率
- `max_time_slice` 在 `sched_init()` 中由框架设置，这里不处理

关键点：

- 使用 `list_init()` 而不是简单的赋值 `NULL`，因为 ucore 使用的是带哨兵节点的双向循环链表

2.2.2 RR_enqueue 函数实现

```
static void
RR_enqueue(struct run_queue *rq, struct proc_struct *proc)
{
    assert(list_empty(&(proc->run_link)));

    // 将进程添加到运行队列尾部
    list_add_before(&(rq->run_list), &(proc->run_link));

    // 如果进程的时间片为0或超过最大值，重置为最大时间片
    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice) {
        proc->time_slice = rq->max_time_slice;
    }

    // 设置进程所属的运行队列
    proc->rq = rq;

    // 增加队列中的进程计数
    rq->proc_num++;
}
```

设计思路：

1. **插入位置的选择**：使用 `list_add_before(&(rq->run_list), ...)` 将进程添加到队列尾部。这是因为 `run_list` 是哨兵节点，其前面就是队列尾部。
2. **时间片的处理**：新进程或时间片耗尽的进程入队时，需要重置时间片设置为 `max_time_slice`，保证公平性。
3. **分支的处理**：
 1. 分支 1 (`proc->time_slice == 0`) 覆盖两种场景：① 新创建的进程（初始时间片为 0）；② 上一次运行时时间片耗尽的进程（被抢占后重新入队）。这两种情况都需要重置时间片为最大值，保证进程获得完整的时间片。
 2. 分支 2 (`proc->time_slice > rq->max_time_slice`) 防御性编程，避免因异常操作（如手动修改时间片）导致单个进程占用 CPU 过久，破坏 RR 算法的公平性。
4. **重置时间片是 RR 算法公平性的核心保障**：所有就绪进程的时间片长度一致，不会出现某进程时间片更长、抢占更多 CPU 的情况。

边界条件处理：断言检查 `proc->run_link` 是否为空，防止重复入队时间片为 0 时重置，处理新创建进程的情况。

2.2.3 RR_dequeue 函数实现

```
static void
RR_dequeue(struct run_queue *rq, struct proc_struct *proc)
{
    // LAB6: YOUR CODE
    assert(!list_empty(&(proc->run_link)) && proc->rq == rq);

    // 从运行队列中删除进程
    list_del_init(&(proc->run_link));

    // 减少队列中的进程计数
    rq->proc_num--;
}
```

设计思路:

1. 断言检查:

1. 第一个条件 (`!list_empty`) 确保进程的 `run_link` 节点已挂载到链表中 (非空), 防止删除未入队的进程导致链表结构破坏 (如修改不存在的节点指针)。
2. 第二个条件 (`proc->rq == rq`) 确保进程所属的就绪队列是当前操作的队列, 避免跨 CPU / 跨队列的错误操作。

2. 使用 `list_del_init`:

1. `list_del(1e)` 的核心逻辑是将节点从链表中移除, 但仅执行 `list_del` 会导致节点的 `prev/next` 指针指向原链表的节点, 形成悬空指针——若后续通过 `list_empty` 判断进程是否在队列中, 会得到错误结果。
2. `list_del_init` 在 `list_del` 的基础上, 额外调用 `list_init(1e)` 将节点重置为“空”状态即 `prev/next` 指向自身, 既完成删除, 又保证节点状态的一致性, 后续可通过 `list_empty` 正确判断进程是否在队列中。

关键点: `list_del_init` 比 `list_del` 更安全, 避免悬空指针

2.2.4 RR_pick_next 函数实现

```
static struct proc_struct *
RR_pick_next(struct run_queue *rq)
{
    // 获取队列头部的元素
    list_entry_t *le = list_next(&(rq->run_list));

    // 如果队列为空, 返回NULL
    if (le == &(rq->run_list)) {
        return NULL;
    }

    // 通过链表节点获取进程结构体
    return le2proc(le, run_link);
}
```

设计思路:

1. **获取队首进程**: `list_next(&(rq->run_list))` 获取哨兵节点的下一个节点。即队列中的第一个进程，由于 `run_list` 是双向循环链表的哨兵，遍历队列需从 `list_next(run_list)` 开始，到 `list_prev(run_list)` 结束。
2. **空队列检查**: 如果队列为空，`list_next` 返回的是哨兵节点本身检查 `le == &(rq->run_list)` 判断队列是否为空
3. **核心转换宏**: `le2proc(le, member)` 是核心转换宏，底层实现为：

```
#define le2proc(le, member) \
    (struct proc_struct *)((char *)le - offsetof(struct proc_struct, member))
```

- 根据链表节点 `le` 的地址，以及该节点在 `proc_struct` 中的成员名 `run_link`，通过 `offsetof` 获取 `run_link` 的偏移量，反向计算出 `proc_struct` 的起始地址。

边界条件处理: 空队列时返回 `NULL`，由调度器框架处理（使用 `idleproc`）

2.2.5 RR_proc_tick 函数实现

```
static void
RR_proc_tick(struct run_queue *rq, struct proc_struct *proc)
{
    // LAB6: YOUR CODE
    if (proc->time_slice > 0) {
        // 时间片减一
        proc->time_slice--;
    }
    if (proc->time_slice == 0) {
        // 时间片耗尽，设置需要调度标志
        proc->need_resched = 1;
    }
}
```

设计思路:

1. **时间片递减**: 先判断 `proc->time_slice > 0`，防止时间片被无限递减。每次时钟中断减少 1 个时间片单位，对应进程占用 CPU 的时间减少一个时钟周期，直到时间片耗尽。
2. **设置调度标志**: 时间片耗尽时设置 `need_resched = 1`，触发调度器在适当时机进行进程切换

为什么需要设置 need_resched:

时钟中断处理程序运行在**中断上下文**，此时直接调用 `schedule()` 会导致死锁或上下文混乱。因此采用**“延迟调度”策略**——仅设置标志位，待中断处理完成、返回用户态前的安全点，再检查该标志位，若为 1 则调用 `schedule()` 进行进程切换。

2.4 Round Robin 调度算法的优缺点分析

2.4.1 优点

1. **强公平性，无进程饥饿问题**

RR 算法的公平性体现在 CPU 时间的“机会均等”：所有就绪进程按入队顺序轮流获得长度相同的时间片，且时间片耗尽后仅被放回队列尾部而非剥夺调度资格，从机制上杜绝了“进程饥饿”。

从 ucore 实现来看：RR_enqueue 会为所有入队进程重置时间片为 max_time_slice，RR_pick_next 始终选取队首进程，即使长作业也不会抢占短作业的调度机会，短作业不会因长作业持续执行而无法获得 CPU；

2. 实现简单，易维护且容错性高

RR 算法的核心依赖“FIFO 就绪队列 + 时间片计数”，无需复杂的优先级计算、进程分类或运行时间预估，代码逻辑极简，仅需实现 5 个核心函数（初始化、入队、出队、选下一个进程、时间片更新），核心操作均为双向循环链表的基础增删查，代码量不足百行，调试和维护成本低；

对比复杂度：相较于多级反馈队列（MLFQ）、最短剩余时间优先（SRTF）等算法，RR 无嵌套队列、优先级动态调整等逻辑，出现 bug 的概率极低。

3. 响应时间可预测，适配交互式系统

RR 算法的最坏响应时间可通过公式精准计算：最坏响应时间 = 时间片长度 × 就绪队列进程数。

交互式系统适配性：对于终端、GUI 等交互式场景，用户操作需要快速响应（，通过合理设置时间片，即使就绪进程数为 5，最坏响应时间也仅 100ms，满足交互体验要求；

4. 无进程运行时间依赖，通用性强

RR 算法无需预知进程的 CPU 突发时间，对短作业、长作业、CPU 密集型、I/O 密集型进程均适配，SJF 需要预估进程运行时间，预估误差会导致调度公平性失效；

2.4.2 缺点

1. 上下文切换开销随时间片缩小呈指数增长

上下文切换是 RR 算法的核心开销，每次切换需要完成：保存当前进程寄存器状态→刷新 TLB（快表）→切换页表→恢复目标进程寄存器状态。若时间片设为 1ms，就绪进程数为 10，则每秒切换 1000 次，切换开销占比可能超过 10%；若时间片缩小至 0.1ms，切换次数增至 10000 次 / 秒，开销占比可能超过 50%，有效 CPU 时间被大量消耗；

2. 无优先级机制，无法满足差异化调度需求

RR 算法将所有进程视为“同等重要”，但实际系统中进程存在优先级差异如内核进程 > 用户进程、实时进程 > 普通进程：若系统中同时存在“键盘输入处理进程”（交互式、高优先级）和“大数据计算进程”（后台、低优先级），RR 会为两者分配相同时间片，导致键盘输入响应延迟，影响用户体验；

ucore 纯 RR 实现中，即使手动设置进程优先级，也无法改变调度顺序，优先级字段仅为预留，无实际调度作用。

3. 平均周转时间偏高，对短作业不友好

周转时间是“进程创建到结束的总时间”，RR 算法的平均周转时间显著高于 SJF，核心原因是短作业可能被长作业的时间片“打断”，需要等待多轮调度

4. 缓存局部性失效，降低 CPU 整体性能

CPU 缓存的效率依赖“局部性原理”：进程持续执行时，常用的指令和数据会被缓存，访问速度提升 10~100 倍；而 RR 的频繁切换会破坏局部性

2.5 时间片大小的优化

2.5.1 时间片过大

- 核心优点：上下文切换次数极少，切换开销占比 $< 1\%$ ，CPU 有效时间利用率接近 100% ；
- 核心缺点：
 - 响应时间急剧增加：若就绪进程数为 10，时间片设为 200ms，最坏响应时间 = $200 \times 10 = 2000\text{ms}$ （2 秒），远超交互式系统的 100ms 响应要求，用户操作会出现明显卡顿；
 - 退化为 FCFS：当时间片大于绝大多数进程的 CPU 突发时间时，进程会在一个时间片内完成执行，RR 的轮转特性消失，等价于先来先服务（FCFS），失去对短作业的响应优势。

2.5.2 时间片过小

- 核心优点：响应时间极短，，同时运行多个终端窗口，无明显卡顿；
- 核心缺点：
 - 上下文切换开销主导 CPU 时间：如时间片设为 5ms，就绪进程数为 10，每秒切换 200 次，若单次切换开销为 $5\mu\text{s}$ ，总切换开销 = $200 \times 5\mu\text{s} = 1\text{ms}$ / 秒，占比 20%（5ms 时间片下，每秒有效执行时间仅 4ms）；
 - 吞吐量下降：吞吐量是“单位时间完成的进程数”，过小的时间片导致进程有效执行时间被压缩，单位时间内完成的进程数显著减少；
 - 缓存失效加剧：切换频率越高，缓存未命中率越高，CPU 流水线频繁中断，进一步降低执行效率。

2.5.3 最佳时间片选择

(1) 经典经验法则（静态优化）

最佳时间片需满足：时间片长度 $\approx$ 一次典型用户交互的 CPU 处理时间 + 上下文切换开销

- 行业通用取值：桌面 / 交互式系统为 10~100ms（Linux 默认 200ms，可通过 `/proc/sys/kernel/sched_rr_timeslice_ms` 调整），嵌入式系统为 1~10ms（资源受限，需平衡响应和开销）；
- ucore 中的实操：找到 `MAX_TIME_SLICE` 的定义位置（通常在 `kern/sched/sched.h`），默认值多为 10 或 20；

(2) 动态时间片优化（进阶策略）

静态时间片无法适配系统负载变化，因此实际系统（如 Linux）采用“动态时间片”策略，可作为 RR 算法的优化方向：

- 基于系统负载调整：低负载增大时间片，减少切换开销；高负载减小时间片，保证响应时间；
- 基于进程类型调整：I/O 密集型进程分配小时间片，利用其频繁阻塞的特性，减少切换开销；CPU 密集型进程分配大时间片，提升缓存利用率和有效执行时间；

2.6 扩展思考

2.6.1 优先级RR调度的实现

要实现优先级RR调度，需要修改以下部分：

1. 数据结构修改：

```
struct run_queue {
    list_entry_t run_list[NUM_PRIORITY]; // 多个优先级队列
    unsigned int proc_num;
    int max_time_slice;
};
```

2. enqueue修改:

```
static void priority_RR_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    int prio = proc->priority;
    list_add_before(&(rq->run_list[prio]), &(proc->run_link));
    // ...
}
```

3. pick_next修改:

```
static struct proc_struct *priority_RR_pick_next(struct run_queue *rq) {
    for (int i = 0; i < NUM_PRIORITY; i++) {
        if (!list_empty(&(rq->run_list[i]))) {
            return le2proc(list_next(&(rq->run_list[i])), run_link);
        }
    }
    return NULL;
}
```

2.6.2 多核调度支持

当前实现**不支持**多核调度，因为目前只有一个全局运行队列，没有CPU亲和性管理且缺少多核间的负载均衡

改进方案:

1. **每CPU运行队列**: struct run_queue per_cpu_rq[NR_CPUS];
2. **负载均衡器**: 定期检查各CPU队列负载，将进程从繁忙CPU迁移到空闲CPU
3. **自旋锁保护**: spinlock_t rq_lock[NR_CPUS];
4. **CPU亲和性**: 进程可以指定偏好的CPU，减少缓存迁移开销

2.7 结果分析

make qemu 结果

```

++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 1
set priority to 2
set priority to 3
set priority to 4
set priority to 5
child pid 7, acc 264000, time 2010
child pid 3, acc 292000, time 2020
child pid 4, acc 288000, time 2030
child pid 5, acc 280000, time 2040
child pid 6, acc 272000, time 2050
main: pid 0, acc 292000, time 2050
main: pid 4, acc 288000, time 2050
main: pid 5, acc 280000, time 2060
main: pid 6, acc 272000, time 2060
main: pid 7, acc 264000, time 2060
main: wait pids over
sched result: 1 1 1 1 1
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:569:
    initproc exit.

```

www@www-VMware-Virtual-Platform: ~/riscv/lab6\$

```

sched class: RR_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 1
set priority to 2
set priority to 3
set priority to 4
set priority to 5
child pid 7, acc 264000, time 2010
child pid 3, acc 292000, time 2020
child pid 4, acc 288000, time 2030
child pid 5, acc 280000, time 2040
child pid 6, acc 272000, time 2050
main: pid 0, acc 292000, time 2050
main: pid 4, acc 288000, time 2050
main: pid 5, acc 280000, time 2060
main: pid 6, acc 272000, time 2060
main: pid 7, acc 264000, time 2060
main: wait pids over
sched result: 1 1 1 1 1
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:569:
    initproc exit.

```

结果分析:

(1) 调度环境准备

- `sched class: RR_scheduler` 明确当前 ucore 内核启用的是时间片轮转调度算法
- `++ setup timer interrupts` 表明内核已完成定时器中断的初始化——RR 算法依赖定时器中断实现“时间片递减”和“调度触发”，这是 RR 算法能按固定时间片轮转的硬件基础。

(2) 用户进程执行阶段

1. 字段含义说明

- 1. `pid`：进程唯一标识，父进程为 `pid=2`（`priority` 进程），子进程为 `pid=3/4/5/6/7`；
- 2. `acc`：进程累计执行的“工作单元”（如循环次数、指令数），反映进程实际占用 CPU 的总时间；
- 3. `time`：系统时间戳，记录进程完成执行的时间点。

2. RR调度的具体体现

- 1. **进程创建与就绪队列入队**：父进程（`pid=2`）通过 `fork` 创建了 5 个子进程（`pid=3~7`），每个子进程被创建后会调用 `RR_enqueue` 加入就绪队列尾部，并重置时间片为最大值。输出中 `main: fork ok,now need to wait pids.` 表明所有子进程已成功入队，等待调度执行
- 2. **轮转执行**：输出中子进程的 `time` 字段依次递增，差值恰好等于 RR 算法的时间片长度（10 个时钟周期），直接体现 RR 算法的“轮转执行”特性
- 3. **公平性**：5 个子进程的 `acc` 值分别为：292000（`pid3`）、288000（`pid4`）、280000（`pid5`）、272000（`pid6`）、264000（`pid7`），数值高度接近（差值仅 8000 左右，占比约 2%-3%），这是 RR 算法“公平调度”的核心体现
- 4. **优先级设置无影响**：输出中多次出现 `set priority to x`（`X=1~6`），但 RR 算法本身**不基于优先级调度**，因此优先级设置未改变调度结果——所有子进程的 `acc` 仍保持均匀，验证了当前启用的是“纯时间片轮转”调度，无优先级加权逻辑。

make grade结果

```
www@www-VMware-Virtual-Platform:~/riscv/lab6$ make grade
priority: (3.6s)
-check result: OK
-check output: OK
Total Score: 50/50
www@www-VMware-Virtual-Platform:~/riscv/lab6$
```

最终得分50/50，表明我们成功实现了RR调度算法！

扩展练习 Challenge 1: 实现 Stride Scheduling 调度算法（需要编码）

一、Stride Scheduling 算法原理

1.1 算法概述

Stride Scheduling 是一种确定性的比例共享（Proportional-Share）调度算法，由 Waldspurger 和 Weihl 于 1995 年提出。与传统的优先级调度不同，Stride 调度通过为每个进程维护一个动态的“通行值”（pass），并在每次调度时选择通行值最小的进程执行，从而实现对 CPU 时间的精确按比例分配。该算法具有确定性、公平性好和可预测性强等特点，适用于需要保证服务份额的实时或公平调度场景。

1.2 核心概念

概念	定义	作用
tickets	进程持有的"票数"，即优先级	表示进程应获得的 CPU 时间份额
stride	步长， $stride = BIG_STRIDE / tickets$	每次调度后增加的值，优先级越高步长越小

概念	定义	作用
pass	当前累计值 (lab6_stride)	记录进程已获得的调度"进度"

1.3 调度策略

- 每次调度时，选择 pass 值最小的进程执行
- 被选中的进程执行后，其 pass 值增加 stride
- 由于高优先级进程的 stride 较小，pass 增长较慢，因此会被更频繁地调度

1.4 BIG_STRIDE 的选取

```
#define BIG_STRIDE 0x7FFFFFFF // 2^31 - 1
```

选择 0x7FFFFFFF 的原因：

- stride 使用 32 位无符号整数存储，会发生溢出
- 比较两个 stride 时使用有符号整数差值：(int32_t)(a - b)
- 为保证溢出时比较仍正确，任意两进程的 stride 差值不能超过 $2^{31} - 1$
- 由于 `pass_max = BIG_STRIDE`（当 priority = 1 时），选择 0x7FFFFFFF 可获得最大精度

二、设计实现过程

2.1 数据结构选择

我们在实现中使用了**斜堆**作为优先队列：

```
#define USE_SKEW_HEAP 1
```

选择原因分析：

Stride调度需要在每次调度时快速找到pass值最小的进程，斜堆作为可合并堆的一种，能够高效支持这一需求：

- **插入与删除复杂度：**均为 $O(\log n)$ ，满足调度器性能要求
- **获取最小值：**堆顶即为最小元素，复杂度 $O(1)$
- **实现优势：**
 - 无需复杂的平衡操作，实现简洁
 - 满足Stride调度频繁插入、删除和获取最小值的操作模式
 - 相比链表实现（ $O(n)$ 查找）显著提升效率

2.2 比较函数实现

```
static int
proc_stride_comp_f(void *a, void *b)
{
    struct proc_struct *p = le2proc(a, lab6_run_pool);
    struct proc_struct *q = le2proc(b, lab6_run_pool);
    int32_t c = p->lab6_stride - q->lab6_stride;
    if (c > 0)
        return 1;
    else if (c == 0)
        return 0;
    else
        return -1;
}
```

实现分析：

该比较函数是斜堆数据结构的核心回调函数，用于确定两个进程在堆中的相对位置。函数首先通过 `le2proc` 宏将通用指针转换为进程控制块指针，然后计算两个进程 `stride` 值的差值进行比较。

本实现的关键技巧在于溢出处理机制。由于 `stride` 值会随着进程的持续执行而不断累加，最终可能超出 32 位无符号整数的表示范围。为解决这一问题，函数将差值显式转换为 `int32_t` 有符号类型，利用二进制补码的数学特性来正确判断大小关系。

- **类型转换原理：**当两个无符号数相减后转为有符号数时，若实际差值在 $[-2^{31}, 2^{31}-1]$ 范围内，补码表示能够正确反映其正负性
- **安全性保证：**配合 `BIG_STRIDE =  $2^{31}-1$` 的设计约束，任意两个就绪进程的 `stride` 差值必然不超过此范围，从而确保比较结果的正确性
- **返回值规范：**返回 1、0、-1 三种状态，符合标准比较函数的接口约定，便于斜堆进行堆序维护

2.3 stride_init - 初始化运行队列

```
static void
stride_init(struct run_queue *rq)
{
    list_init(&(rq->run_list));
    rq->lab6_run_pool = NULL;
    rq->proc_num = 0;
}
```

实现分析：

初始化函数负责将运行队列重置为初始状态，为后续的进程调度操作建立干净的运行环境。该函数在系统启动阶段被调用，确保调度器各数据结构处于一致的初始状态。

函数采用双模式初始化策略，同时对链表和斜堆两种数据结构进行初始化。这种设计使得算法实现可以通过编译宏 `USE_SKEW_HEAP` 在两种模式间灵活切换，便于性能对比测试和调试验证。

- **链表初始化：**调用 `list_init` 将链表头节点的前驱和后继指针均指向自身，形成空的循环双向链表
- **斜堆初始化：**将堆根指针置为 `NULL`，表示当前堆中没有任何节点
- **计数器清零：**将进程计数 `proc_num` 设为 0，准确反映队列的空状态

2.4 stride_enqueue - 进程入队

```
static void
stride_enqueue(struct run_queue *rq, struct proc_struct *proc)
{
    #if USE_SKEW_HEAP
        rq->lab6_run_pool = skew_heap_insert(
            rq->lab6_run_pool,
            &(proc->lab6_run_pool),
            proc_stride_comp_f
        );
    #else
        list_add_before(&(rq->run_list), &(proc->run_link));
    #endif
    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice) {
        proc->time_slice = rq->max_time_slice;
    }
    proc->rq = rq;
    rq->proc_num++;
}
```

实现分析：

入队函数在进程状态转变为就绪态时被调用，负责将进程加入运行队列并完成相关的状态维护工作。该函数是进程生命周期管理的关键环节，直接影响调度器的正确运行。

在斜堆模式下，函数调用 `skew_heap_insert` 将进程节点插入堆中。斜堆的插入操作会自动根据比较函数的结果调整堆结构，确保堆顶始终是 stride 值最小的进程。插入操作的平摊时间复杂度为 $O(\log n)$ ，相比链表的 $O(1)$ 插入稍慢，但换取了选择操作的高效性。

- **堆插入机制：** `skew_heap_insert` 返回新的堆根指针，因为插入操作可能改变堆顶元素
- **时间片初始化：** 对于时间片为 0 或超出最大限制的进程，将其重置为队列的默认最大时间片，保证进程获得公平的执行机会
- **队列关联建立：** 将进程的 `rq` 指针指向当前运行队列，建立进程与队列的双向关联关系
- **计数同步更新：** 递增 `proc_num` 计数器，确保队列状态信息的准确性

2.5 stride_dequeue - 进程出队

```
static void
stride_dequeue(struct run_queue *rq, struct proc_struct *proc)
{
    #if USE_SKEW_HEAP
        rq->lab6_run_pool = skew_heap_remove(
            rq->lab6_run_pool,
            &(proc->lab6_run_pool),
            proc_stride_comp_f
        );
    #else
        assert(!list_empty(&(proc->run_link)));
        list_del_init(&(proc->run_link));
    #endif
    rq->proc_num--;
}
```

实现分析：

出队函数在进程被选中执行或因其他原因离开就绪队列时被调用，负责将指定进程从运行队列中移除。与入队操作相对应，出队操作同样需要维护数据结构的完整性和状态的一致性。

斜堆的删除操作相比普通堆更为高效。skew_heap_remove 函数能够删除堆中的任意节点（不仅限于堆顶），这在进程因优先级变化或其他原因需要提前移除时非常有用。删除操作通过合并被删节点的左右子树来完成，平摊时间复杂度同样为 $O(\log n)$ 。

- **精确删除能力**：斜堆支持删除任意指定节点，而非仅能删除堆顶，提供了更大的操作灵活性
- **堆根更新**：删除操作可能导致堆根变化，因此需要用返回值更新 lab6_run_pool 指针
- **链表安全检查**：在链表模式下，通过断言确保待删除节点确实在队列中，防止非法操作导致的数据结构损坏
- **计数同步递减**：及时更新 proc_num，保持队列元信息的准确性

2.6 stride_pick_next - 选择下一个进程

```
static struct proc_struct *
stride_pick_next(struct run_queue *rq)
{
    #if USE_SKEW_HEAP
        if (rq->lab6_run_pool == NULL) {
            return NULL;
        }
        struct proc_struct *p = le2proc(rq->lab6_run_pool, lab6_run_pool);
    #else
        list_entry_t *le = list_next(&(rq->run_list));
        if (le == &(rq->run_list)) {
            return NULL;
        }
        struct proc_struct *p = le2proc(le, run_link);
        le = list_next(le);
        while (le != &(rq->run_list)) {
            struct proc_struct *q = le2proc(le, run_link);
            if ((int32_t)(q->lab6_stride - p->lab6_stride) < 0) {
                p = q;
            }
            le = list_next(le);
        }
    #endif
    if (p->lab6_priority == 0) {
        p->lab6_stride += BIG_STRIDE;
    } else {
        p->lab6_stride += BIG_STRIDE / p->lab6_priority;
    }
    return p;
}
```

实现分析：

选择函数是 Stride 调度算法的核心，负责从就绪队列中选出下一个应当执行的进程。该函数实现了 Stride 算法的核心思想：始终选择当前 stride 值最小的进程，以实现 CPU 时间的按比例公平分配。

在斜堆模式下，由于堆顶元素必然是 stride 值最小的进程，选择操作仅需 $O(1)$ 的时间复杂度即可完成。而在链表模式下，则需要遍历整个队列进行线性查找，时间复杂度为 $O(n)$ 。这一性能差异是选择斜堆作为主要数据结构的重要原因。

- **空队列处理**：函数首先检查队列是否为空，若为空则返回 NULL，表示当前没有可调度的进程
- **最小元素获取**：斜堆直接读取堆顶；链表则通过遍历比较找出 stride 最小的进程
- **stride 更新公式**：选中进程后执行 `stride += BIG_STRIDE / priority`，优先级越高（数值越大），增量越小，使得高优先级进程能更频繁地被选中
- **优先级零处理**：当优先级为 0 时直接加 `BIG_STRIDE`，既避免了除零错误，也使该进程在下一轮调度中处于最低优先地位

stride 更新机制体现了算法的比例共享特性：假设两个进程优先级分别为 2 和 4，则它们的 stride 增量比为 2:1，经过足够长的时间后，两者获得的 CPU 时间比将趋近于 1:2，即与优先级成正比。

2.7 stride_proc_tick - 时钟中断处理

```
static void
stride_proc_tick(struct run_queue *rq, struct proc_struct *proc)
{
    if (proc->time_slice > 0) {
        proc->time_slice--;
    }
    if (proc->time_slice == 0) {
        proc->need_resched = 1;
    }
}
```

实现分析：

时钟中断处理函数在每次定时器中断时被调用，负责更新当前运行进程的时间片状态并在必要时触发重新调度。该函数是操作系统抢占式调度机制的基础，确保任何进程都不能无限占用 CPU。

函数的实现逻辑与轮转调度器完全相同，这体现了时间片机制与具体调度策略之间的正交性设计。Stride 算法通过 stride 值决定下一个运行的进程，而时间片机制则控制每个进程单次运行的最大时长，两者相互独立又协同工作。

- **时间片递减**：每次时钟中断将当前进程的 `time_slice` 减 1，精确计量进程的实际运行时间
- **边界条件判断**：仅当时间片大于 0 时才执行递减，避免无符号整数下溢
- **重调度触发**：时间片耗尽时设置 `need_resched` 标志为 1，通知调度器在合适的时机进行进程切换
- **抢占式保障**：即使是高优先级进程，用完时间片后也必须让出 CPU，防止低优先级进程饥饿

这种设计使得 Stride 调度器既保持了比例共享的公平性，又具备了抢占式调度的响应性，是一种兼顾效率与公平的调度方案。

2.8 切换调度器 (sched.c)

```
void sched_init(void)
{
    list_init(&timer_list);

    // LAB6 CHALLENGE 1: YOUR CODE
    // 切换调度算法：将 default_sched_class 改为 stride_sched_class
```

```

// 使用 RR 调度器:
// sched_class = &default_sched_class;
// 使用 stride 调度器:
sched_class = &stride_sched_class;

rq = &__rq;
rq->max_time_slice = MAX_TIME_SLICE;
sched_class->init(rq);

printf("sched class: %s\n", sched_class->name);
}

```

三、实验结果

3.1 运行输出

```

sched class: stride_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
child pid 7, acc 280000, time 2010
child pid 6, acc 252000, time 2020
child pid 5, acc 196000, time 2020
child pid 4, acc 160000, time 2030
child pid 3, acc 88000, time 2040
main: pid 3, acc 88000, time 2040
main: pid 4, acc 160000, time 2050
main: pid 5, acc 196000, time 2050
main: pid 6, acc 252000, time 2050
main: pid 0, acc 280000, time 2050
main: wait pids over
sched result: 1 2 2 3 3
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:569:
initproc exit.

```

```
sched class: stride_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
child pid 7, acc 280000, time 2010
child pid 6, acc 252000, time 2020
child pid 5, acc 196000, time 2020
child pid 4, acc 160000, time 2030
child pid 3, acc 88000, time 2040
main: pid 3, acc 88000, time 2040
main: pid 4, acc 160000, time 2050
main: pid 5, acc 196000, time 2050
main: pid 6, acc 252000, time 2050
main: pid 0, acc 280000, time 2050
main: wait pids over
sched result: 1 2 2 3 3
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:569:
initproc exit.
```

3.2 结果分析

根据运行输出，整理各进程的执行数据如下：

进程 PID	优先级 (priority)	执行计数 (acc)	相对比例
pid 7	6	280000	3.18
pid 6	5	252000	2.86
pid 5	4	196000	2.23
pid 4	3	160000	1.82
pid 3	2	88000	1

以 pid 3 (priority=2) 为基准，各进程的执行次数比例为：

- **实际比例：** 1.00 : 1.82 : 2.23 : 2.86 : 3.18
- **理论比例（优先级比）：** 1.00 : 1.50 : 2.00 : 2.50 : 3.00

分析：实际执行比例与理论优先级比例基本吻合，高优先级进程获得了更多的 CPU 时间。存在的轻微偏差可能由以下因素导致：

1. 进程创建和退出的时间差异
2. 时钟中断的离散性
3. stride 值溢出处理的影响

总体而言，实验结果验证了 Stride 调度算法能够按优先级比例公平分配 CPU 时间。

四、Stride 算法正比性证明

4.1 问题陈述

证明：经过足够多的时间片后，每个进程分配到的时间片数目与其优先级成正比。

4.2 证明过程

设有 n 个进程，进程 i 的优先级为 P_i ，步长为 $S_i = \frac{BIGSTRIDE}{P_i}$ 。

引理：在任意时刻，任意两个进程的 stride 差值有界。

由于每次调度选择 stride 最小的进程，设最大 stride 为 $stride_{max}$ ，最小为 $stride_{min}$ ，则：

$$stride_{max} - stride_{min} \leq \max(S_i) = \frac{BIGSTRIDE}{\min(P_i)}$$

主定理：设经过 T 个时间片后，进程 i 被调度了 N_i 次。

对于进程 i ，其 stride 增加量为：

$$\Delta stride_i = N_i \times S_i = N_i \times \frac{BIGSTRIDE}{P_i}$$

由于所有进程的 stride 差值有界（设为常数 C ），当 T 足够大时： $|\Delta stride_i - \Delta stride_j| \leq C$

$$\text{即：} \left| N_i \times \frac{BIGSTRIDE}{P_i} - N_j \times \frac{BIGSTRIDE}{P_j} \right| \leq C$$

当 $T \rightarrow \infty$ 时， C 相对于 $\Delta stride$ 可忽略，因此： $N_i \times \frac{1}{P_i} \approx N_j \times \frac{1}{P_j}$

$$\text{即：} \frac{N_i}{N_j} \approx \frac{P_i}{P_j}$$

结论：进程获得的调度次数与其优先级成正比。

五、多级反馈队列调度算法的设计

5.1 算法概述

多级反馈队列调度算法（MLFQ）是一种综合性的进程调度算法，它结合了时间片轮转和优先级调度的优点，能够在不需要预先知道进程运行时间的情况下，同时满足响应时间和周转时间的优化目标。

MLFQ 的核心思想是通过观察进程的实际行为动态调整其优先级：交互式进程因频繁主动让出 CPU 而保持在高优先级队列，获得快速响应；计算密集型进程因持续占用 CPU 而逐渐降低优先级队列，使用更长的时间片以减少上下文切换开销。

5.2 概要设计

5.2.1 数据结构设计

在现有 uCore 框架基础上，需要扩展以下数据结构：

```
// 多级队列数量
#define MLFQ_NUM_QUEUES 4

// 扩展运行队列结构
struct run_queue {
    list_entry_t run_list;           // 原有字段保留
    unsigned int proc_num;          // 总进程数
    int max_time_slice;             // 最大时间片

    // MLFQ 新增字段
    list_entry_t mlfq_queues[MLFQ_NUM_QUEUES]; // 多级队列数组
```

```
int mlfq_time_slices[MLFQ_NUM_QUEUES];           // 各级队列的时间片
unsigned int mlfq_proc_num[MLFQ_NUM_QUEUES];      // 各级队列的进程数
unsigned int mlfq_boost_interval;                 // 优先级提升周期
unsigned int mlfq_ticks;                          // 全局时钟计数
};

// 扩展进程控制块结构
struct proc_struct {
    // ... 原有字段 ...

    // MLFQ 新增字段
    int mlfq_priority;                            // 当前所在队列级别（0 为最高优先级）
    int mlfq_allotment;                           // 当前级别剩余的时间配额
    int mlfq_cpu_ticks;                           // 在当前级别累计使用的 CPU 时间
};
```

5.2.2 调度规则设计

MLFQ 遵循以下五条基本规则：

规则编号	规则描述
规则 1	若 $Priority(A) > Priority(B)$ ，则运行 A
规则 2	若 $Priority(A) = Priority(B)$ ，则 A 与 B 按时间片轮转运行
规则 3	新进程进入系统时，置于最高优先级队列
规则 4	进程在当前级别用完时间配额后，降至下一级队列
规则 5	经过一定时间周期 S 后，将所有进程提升至最高优先级队列

5.2.3 时间片配置策略

各级队列采用递增的时间片长度，典型配置如下：

队列级别	优先级	时间片	适用进程类型
队列 0	最高	1 个时钟周期	交互式进程
队列 1	次高	2 个时钟周期	混合型进程
队列 2	次低	4 个时钟周期	一般计算进程
队列 3	最低	8 个时钟周期	CPU 密集型进程

5.3 详细设计

5.3.1 mlfq_init - 初始化函数

```
static void
mlfq_init(struct run_queue *rq)
{
    // 初始化各级队列
    for (int i = 0; i < MLFQ_NUM_QUEUES; i++) {
```

```

        list_init(&(rq->mlfq_queues[i]));
        rq->mlfq_proc_num[i] = 0;
    }

    // 设置各级队列的时间片（指数递增）
    rq->mlfq_time_slices[0] = 1;
    rq->mlfq_time_slices[1] = 2;
    rq->mlfq_time_slices[2] = 4;
    rq->mlfq_time_slices[3] = 8;

    // 设置优先级提升周期（例如每 100 个时钟周期）
    rq->mlfq_boost_interval = 100;
    rq->mlfq_ticks = 0;

    rq->proc_num = 0;
}

```

设计说明：

初始化函数负责建立多级队列的初始状态。每个队列使用独立的双向循环链表实现，便于进程的插入和删除操作。时间片采用指数递增策略，使得低优先级队列的进程能够获得更长的连续执行时间，减少上下文切换的开销。

优先级提升周期是防止饥饿的关键参数。该值的选择需要权衡：过小会导致 CPU 密集型进程频繁回到高优先级队列，影响交互式进程的响应性；过大则可能导致低优先级进程长时间得不到执行。

5.3.2 mlfq_enqueue - 进程入队

```

static void
mlfq_enqueue(struct run_queue *rq, struct proc_struct *proc)
{
    int priority = proc->mlfq_priority;

    // 确保优先级在有效范围内
    if (priority < 0) {
        priority = 0;
    } else if (priority >= MLFQ_NUM_QUEUES) {
        priority = MLFQ_NUM_QUEUES - 1;
    }
    proc->mlfq_priority = priority;

    // 将进程加入对应优先级队列的末尾
    list_add_before(&(rq->mlfq_queues[priority]), &(proc->run_link));

    // 设置时间片为当前队列的配额
    if (proc->time_slice == 0 || proc->time_slice > rq->mlfq_time_slices[priority]) {
        proc->time_slice = rq->mlfq_time_slices[priority];
    }

    // 更新队列关联和计数
    proc->rq = rq;
    rq->mlfq_proc_num[priority]++;
    rq->proc_num++;
}

```

设计说明：

入队函数根据进程当前的优先级将其插入对应的队列。采用尾插法（FIFO）确保同一队列内的进程按到达顺序执行，实现公平的时间片轮转。

时间片的设置采用条件赋值策略：仅当进程的时间片为零或超出当前队列限制时才重置。这种设计允许被 I/O 中断的进程在返回就绪态时保留剩余时间片，有利于交互式进程获得更好的响应性。

5.3.3 mlfq_dequeue - 进程出队

```
static void
mlfq_dequeue(struct run_queue *rq, struct proc_struct *proc)
{
    int priority = proc->mlfq_priority;

    // 安全检查
    assert(priority >= 0 && priority < MLFQ_NUM_QUEUES);
    assert(!list_empty(&(proc->run_link)));

    // 从链表中移除
    list_del_init(&(proc->run_link));

    // 更新计数
    rq->mlfq_proc_num[priority]--;
    rq->proc_num--;
}
```

设计说明：

出队函数从相应优先级队列中移除进程。通过断言确保操作的合法性，防止对无效节点的删除操作导致链表结构损坏。`list_del_init` 在删除节点的同时将其指针重新初始化，避免悬垂指针问题。

5.3.4 mlfq_pick_next - 选择下一个进程

```
static struct proc_struct *
mlfq_pick_next(struct run_queue *rq)
{
    // 从最高优先级队列开始查找
    for (int i = 0; i < MLFQ_NUM_QUEUES; i++) {
        if (!list_empty(&(rq->mlfq_queues[i]))) {
            // 选择该队列的第一个进程（FIFO）
            list_entry_t *le = list_next(&(rq->mlfq_queues[i]));
            struct proc_struct *proc = le2proc(le, run_link);
            return proc;
        }
    }

    // 所有队列均为空
    return NULL;
}
```

设计说明：

进程选择是 MLFQ 调度的核心逻辑。函数按优先级从高到低遍历各级队列，选择第一个非空队列的队首进程。这种设计直接体现了规则 1（高优先级优先）和规则 2（同优先级轮转）。

算法的时间复杂度为 $O(K)$ ，其中 K 为队列级数。由于 K 通常是一个较小的常数（如 4 或 8），因此可以认为选择操作具有常数时间复杂度。

5.3.5 mlfq_proc_tick - 时钟中断处理

```
static void
mlfq_proc_tick(struct run_queue *rq, struct proc_struct *proc)
{
    // 全局时钟递增
    rq->mlfq_ticks++;

    // 检查是否需要优先级提升（规则 5）
    if (rq->mlfq_ticks >= rq->mlfq_boost_interval) {
        mlfq_priority_boost(rq);
        rq->mlfq_ticks = 0;
    }

    // 时间片递减
    if (proc->time_slice > 0) {
        proc->time_slice--;
        proc->mlfq_cpu_ticks++;
    }

    // 时间片用尽
    if (proc->time_slice == 0) {
        proc->need_resched = 1;

        // 检查是否需要降级（规则 4）
        if (proc->mlfq_priority < MLFQ_NUM_QUEUES - 1) {
            proc->mlfq_priority++;
            proc->mlfq_cpu_ticks = 0; // 重置累计时间
        }
    }
}
```

设计说明：

时钟中断处理函数承担多重职责，是 MLFQ 动态调整机制的核心：

- **全局时钟维护**：记录系统运行的总时钟周期数，用于触发周期性的优先级提升
- **优先级提升检查**：达到提升周期时调用 boost 函数，将所有进程移至最高优先级队列，防止低优先级进程饥饿
- **时间片管理**：递减当前进程的时间片，并累计其在当前级别的 CPU 使用时间
- **降级判断**：时间片用完时将进程降至下一级队列，体现对 CPU 密集型进程的惩罚机制

5.3.6 mlfq_priority_boost - 优先级提升

```
static void
mlfq_priority_boost(struct run_queue *rq)
{
    // 将所有非最高优先级队列的进程移至最高优先级队列
    for (int i = 1; i < MLFQ_NUM_QUEUES; i++) {
        while (!list_empty(&(rq->mlfq_queues[i]))) {
            // 取出进程
        }
    }
}
```

```

list_entry_t *le = list_next(&(rq->mlfq_queues[i]));
struct proc_struct *proc = le2proc(le, run_link);

// 从当前队列移除
list_del_init(&(proc->run_link));
rq->mlfq_proc_num[i]--;

// 重置优先级和时间片
proc->mlfq_priority = 0;
proc->time_slice = rq->mlfq_time_slices[0];
proc->mlfq_cpu_ticks = 0;

// 加入最高优先级队列
list_add_before(&(rq->mlfq_queues[0]), &(proc->run_link));
rq->mlfq_proc_num[0]++;
    }
}
}

```

设计说明：

优先级提升是 MLFQ 防止饥饿的关键机制。该函数周期性地将所有进程提升至最高优先级队列，使得长期运行的 CPU 密集型进程也能获得执行机会。

实现时需要注意：

- 遍历从队列 1 开始，因为队列 0 的进程已是最高优先级
- 每个进程的优先级、时间片和累计 CPU 时间均需重置
- 进程计数需要同步更新，确保各队列状态的一致性

5.4 调度器接口封装

```

struct sched_class mlfq_sched_class = {
    .name = "mlfq_scheduler",
    .init = mlfq_init,
    .enqueue = mlfq_enqueue,
    .dequeue = mlfq_dequeue,
    .pick_next = mlfq_pick_next,
    .proc_tick = mlfq_proc_tick,
};

```

通过标准的 `sched_class` 接口封装，MLFQ 调度器可以无缝集成到 uCore 的调度框架中，与现有的 RR 和 Stride 调度器保持一致的调用方式。

5.5 算法特性分析

特性	说明
自适应调度	无需预先知道进程行为，通过运行时观察自动分类
响应性好	交互式进程保持在高优先级队列，获得快速响应
吞吐量高	CPU 密集型进程使用长时间片，减少上下文切换开销

特性	说明
防止饥饿	周期性优先级提升确保所有进程都能获得执行机会
灵活可调	队列数、时间片、提升周期等参数均可根据场景配置

5.6 与 Stride 算法的对比

对比维度	MLFQ	Stride
调度目标	兼顾响应时间和吞吐量	按比例公平分配 CPU 时间
优先级机制	动态调整（基于行为）	静态设置（基于配置）
适用场景	通用桌面/服务器环境	需要精确比例分配的场景
实现复杂度	较高（多队列、多规则）	较低（单一数据结构）
参数敏感性	高（需调优多个参数）	低（仅需设置优先级）

六、实验总结

6.1 主要收获

- 深入理解了 Stride 调度算法：**掌握了如何通过 stride 和 pass 机制实现精确的比例共享调度。
- 学习了溢出处理技巧：**通过有符号整数差值比较解决了 stride 溢出问题。
- 实践了高效数据结构：**使用斜堆（Skew Heap）将调度复杂度从 $O(n)$ 降低到 $O(\log n)$ 。
- 理解了调度框架设计：**uCore 的 `sched_class` 接口设计使得不同调度算法可以方便切换。

6.2 实验中遇到的问题及解决

问题	原因	解决方案
stride 比较错误	直接使用无符号整数比较	改用有符号整数差值比较
除零错误	priority 可能为 0	添加 <code>priority == 0</code> 的特殊处理
调度器未切换	忘记修改 <code>sched.c</code>	将 <code>sched_class</code> 改为 <code>&stride_sched_class</code>